

# Comunicaciones Digitales - Taller III

Bruno Cardozo  
Comunicaciones Digitales 2026  
Facultad de Ingeniería  
Montevideo, Uruguay  
brunotadeocardozo16@gmail.com

Florencia Roque  
Comunicaciones Digitales 2026  
Facultad de Ingeniería  
Montevideo, Uruguay  
flopa.roque@gmail.com

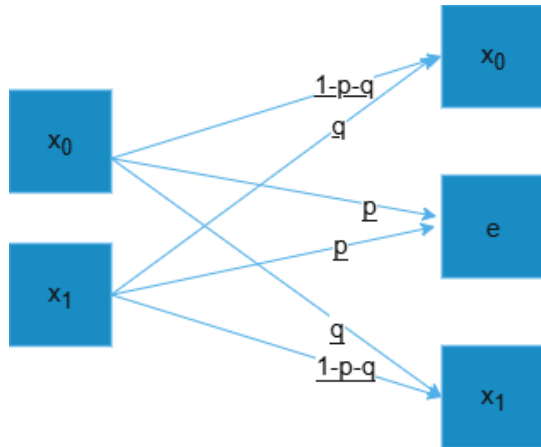


Figura 1. Diagrama del canal simétrico con borrado

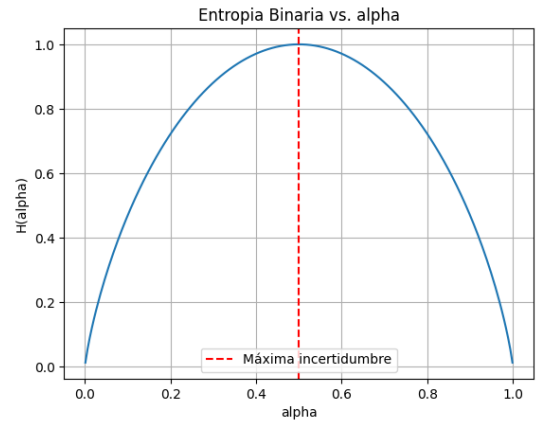


Figura 2. Entropía binaria.

## I. PARTE I - CANAL SIMÉTRICO CON BORRADO

En una primera instancia trabajaremos con un canal binario simétrico con borrado, este canal se trata de una fuente generadora de dos símbolos  $X = \{x_0, x_1\}$  y un receptor que decide sobre un alfabeto  $Y = \{x_0, e, x_1\}$ . El símbolo  $e$  se conoce como “borradura” y el mismo es seleccionado por el receptor cuando no es capaz de decidir con certeza que envió el transmisor. Los símbolos son generados de forma independiente con probabilidades

$$P(X = x_0) = \alpha \quad (1)$$

$$P(X = x_1) = 1 - \alpha \quad (2)$$

y las probabilidades condicionadas de cada símbolo son las siguientes

$$P(Y = x_0|X = x_0) = P(Y = x_1|X = x_1) = 1 - p - q \quad (3)$$

$$P(Y = x_1|X = x_0) = P(Y = x_0|X = x_1) = q \quad (4)$$

$$P(Y = e|X = x_0) = P(Y = e|X = x_1) = p \quad (5)$$

Esto se puede apreciar mejor a través del diagrama que muestra la figura 1.

**Parte (a)** La probabilidad de que un símbolo sea borrado por el algoritmo es la probabilidad de tener un borrado,

ponderado por la probabilidad de aparición del símbolo, y como se trata de símbolos binarios

$$P(Y = e|X) = p \quad (6)$$

**Parte (b)** Los bits de información transmitidos por símbolos vienen dados por la entropía de la fuente, y por tanto

$$H(X) = \sum_{x \in X} p(x) \log \left( \frac{1}{p(x)} \right) = \alpha \log \left( \frac{1}{\alpha} \right) + (1-\alpha) \log \left( \frac{1}{1-\alpha} \right) \quad (7)$$

$$\Rightarrow H(X) = \Omega(\alpha) \quad (8)$$

donde  $\log$  es el logaritmo en base 2, y  $\Omega(\alpha)$  representa la entropía binaria de alfa.

**Parte (c)** En la figura 2 se puede observar cómo la entropía se maximiza cuando los símbolos son equiprobables, es decir  $\alpha = \frac{1}{2}$ . En este caso, la entropía vale  $H(\alpha) = 1$  bit de información por símbolo.

**Parte (d)** Ahora nos preguntamos ¿Es posible modificar un sistema BPSK para que este modelo de canal sea adecuado? Y la respuesta es sí, podemos modificar un receptor BPSK haciendo que decida según las regiones de decisión mostradas en la constelación de la figura 3. Es decir, tendremos una región de decisión a la izquierda de  $-T$  (rojo en la figura) para el símbolo  $x_1$ , una entre  $-T$  y  $T$  (violeta) para el

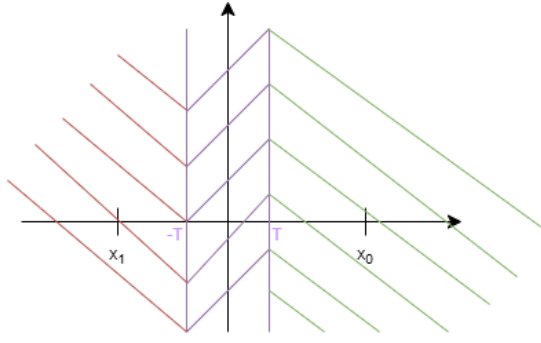


Figura 3. Constelación BPSK. En rojo se marca la región de decisión para el símbolo  $x_1$ , en violeta la del símbolo  $e$  y en verde la del  $x_0$ .

símbolo  $e$  y otra a la derecha de  $T$  (verde) para el símbolo  $x_0$ . Considerando que se trata de un canal que solamente introduce ruido AWGN, calculamos las probabilidades  $p$  y  $q$  en función de los parámetros del sistema. Veamos el caso en el que enviamos el símbolo  $A$ , recordando que en recepción podemos escribir a  $Y_D = X + N_D$

$$p = P(Y = 0|X = -A) = P(-T + A < N_D < T + A) \quad (9)$$

Como se trata de un ruido gaussiano e independiente, estas probabilidades han sido calculadas en laboratorios anteriores, y por tanto

$$p = Q\left(\frac{A - T}{\sigma}\right) - Q\left(\frac{A + T}{\sigma}\right) \quad (10)$$

Planteando una ecuación similar, para el caso en el que recibimos  $A$  dado que enviamos  $-A$ , tenemos

$$q = P(N_D < -T - A) = Q\left(\frac{A + T}{\sigma}\right) \quad (11)$$

**Parte (e)** A partir de éste caso, se tratará con una SNR razonable, por lo que la probabilidad de borratura será pequeña. Esto implica que, las campanas de las distribuciones de los símbolos se estrechan y por tanto la probabilidad de recibir un símbolo erróneo es muchísimo más baja que la de obtener una borratura, y considerando de que ésta última es una probabilidad pequeña, es acertado despreciar la posibilidad de obtener un símbolo a través de éste canal si se trata con una SNR alta.

Al tener una SNR razonable, estamos en el caso de que  $A \gg \sigma$ , entonces como  $T \geq 0$  se cumple que  $A + T \gg \sigma$ , por lo que el área de la cola de la gaussiana que define a  $q$  es despreciable. Por lo tanto, podemos suponer que  $q \approx 0$ .

Despreciar  $q$  implica que obtenemos un nuevo modelo de canal, el mismo se encuentra representado por la figura 4.

**Parte (f)** Dado este nuevo canal, podemos calcular las entropías de la fuente y las condicionadas, además de la información mutua. Comencemos con la salida, la idea intuitiva recae en ¿cuántos bits necesitamos para representar su salida?

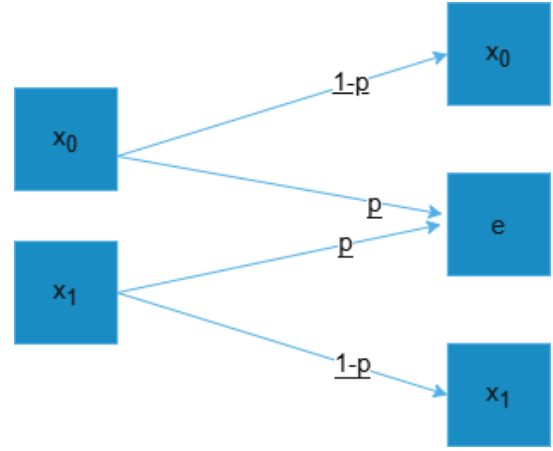


Figura 4. Diagrama del canal simétrico con borrado, despreciando probabilidad de error.

$$H(Y) = \sum_{y \in Y} -P(Y = y) \log(P(Y = y)) \quad (12)$$

$$\begin{aligned} H(Y) &= -\alpha(1-p) \log(\alpha(1-p)) \\ &\quad - (1-\alpha)(1-p) \log((1-\alpha)(1-p)) \\ &\quad - p \log(p) \end{aligned} \quad (13)$$

Desarrollando los logaritmandos y cancelando términos se obtiene que

$$H(Y) = \Omega(p) - (1-p)\Omega(\alpha) = \Omega(p) - (1-p)H(X) \quad (14)$$

Por lo que vemos que, cómo es de esperar, la salida depende tanto de la probabilidad de borrado cómo de la entropía de la fuente.

Ahora nos preguntamos ¿Qué incertidumbre tenemos sobre la salida si conocemos la entrada? Para ello calculemos la entropía asociada

$$H(Y|X) = \sum_{x \in X} P(X = x) H(Y|X = x) \quad (15)$$

$$H(Y|X) = \alpha \Omega(p) + (1-\alpha) \Omega(p) \quad (16)$$

$$H(Y|X) = \Omega(p) \quad (17)$$

La interpretación recae en, dado que conozco cual fue la entrada ¿Qué tanta certeza tengo sobre la salida? y la respuesta es obvia, no sé nada, tengo probabilidad  $1-p$  de acertar o  $p$  de un borrado, por lo que mi incertidumbre recae sobre dicha probabilidad.

Ahora, si sabemos la salida ¿Qué tanta certeza tengo sobre lo que envíe? Intuitivamente podemos decir con precisión que la incertidumbre recae en cuando mi salida es un borrado, porque si sé que mi salida fue un símbolo, se lo que envíe pues

la probabilidad de obtener un símbolo erróneo es despreciable por nuestras hipótesis. Verifiquemos esto mediante cálculos

$$H(X|Y) = \sum_{y \in Y} P(Y = y) H(X|Y = y) \quad (18)$$

$$H(X|Y) = P(Y = e) H(X|Y = e) \quad (19)$$

$$H(X|Y) = p \Omega(\alpha) \quad (20)$$

Una vez obtenidas las correspondientes entropías, es válido preguntarnos ¿Cuánto vale la información mutua entre las variables? Es decir, intuitivamente nos preguntamos ¿Qué tan relacionadas están estas variables?

$$I(X, Y) = H(Y) - H(Y|X) = H(X) - H(X|Y) \quad (21)$$

$$I(X, Y) = (1 - p) H(X) \quad (22)$$

Entonces, vemos que las variables dependen de parámetros del canal y de la fuente, por lo tanto, cómo no podemos controlar el canal, vemos que la única forma de maximizar la relación entre ellos es mejorando la entropía de la fuente. Esto nos lleva a determinar la capacidad del canal, la misma se define cómo la distribución de probabilidad que maximiza la información mutua entre las variables, y por lo último visto, ésta se maximiza cuando la fuente genera bits equiprobables, por tanto

$$C = \max_{p(x)} I(X, Y) = 1 - p \quad (23)$$

**Parte (g)** El sistema con el canal BPSK tradicional tiene dos posibles salidas  $\{x_0, x_1\}$ .

La probabilidad de error de bits es

$$q_{BPSK} = Q\left(\frac{A}{\sigma}\right). \quad (24)$$

La capacidad del canal BPSK es

$$C_{BPSK} = 1 - \Omega(q_{BPSK}). \quad (25)$$

El sistema con el canal BSC-E tiene tres posibles salidas  $\{x_0, e, x_1\}$ .

La probabilidad de error de bits  $q$  es la dada por la ecuación 11 y la probabilidad de borrado  $p$  es la de la ecuación 10.

La capacidad del canal con borrado es

$$C_{BSC-E} = 1 - p + q \log_2 q + (1 - p - q) \log_2 (1 - p - q) + (1 - p) \log_2 (1 - p). \quad (26)$$

En la figura 5 se puede ver un gráfico de las capacidades para el sistema BPSK y el BSC-E para distintos valores de  $A/\sigma$ . Se puede apreciar que a medida que aumenta la SNR ambos canales se comportan bien, en condiciones peores, el sistema tradicional tiene mayor capacidad. Esto es porque hay símbolos que se borran en el sistema con borrado. Sin embargo, la figura 6 muestra que el BER del sistema con borrado

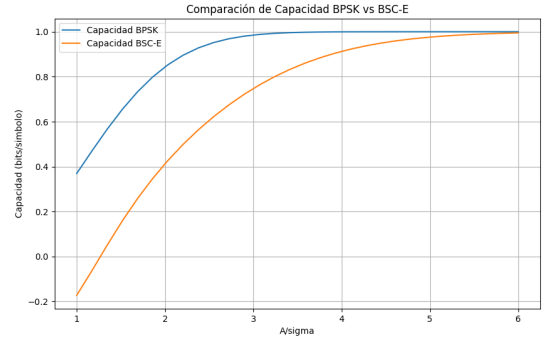


Figura 5. Capacidad sistema BPSK y capacidad sistema con borrados (BSC-E) en función de  $A/\sigma$ . Umbral  $T = A/2$ .

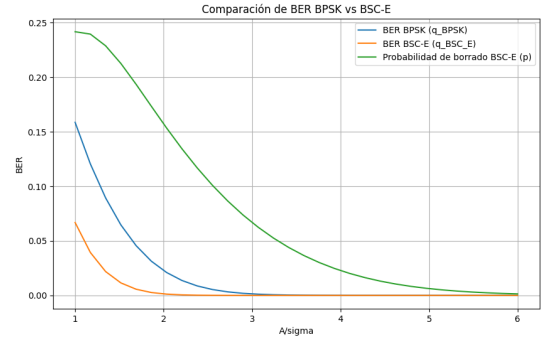


Figura 6. Tasas de errores de bit para sistema BPSK y BSC-E, también se muestra la probabilidad de borrado para el BSC-E, en función de  $A/\sigma$ . Umbral  $T = A/2$ .

es menor que el BER del sistema BPSK en condiciones de SNR malas. Es decir, hay un compromiso entre capacidad y probabilidad de error de bits, se sacrifica algo de capacidad para tener un canal más confiable y robusto. A partir de las simulaciones realizadas, se observa que el sistema con borrado permite gestionar el impacto del ruido AWGN de manera distinta al receptor tradicional. Si bien el sistema con borrado presenta una capacidad inferior, el receptor puede detectar símbolos dudosos (borrados), lo cual es fundamental para sistemas que implementan códigos correctores de errores. Esto queda evidenciado por el hecho de que la probabilidad de error no detectado ( $q$ ) es menor a la probabilidad de borrado ( $p$ ), permitiendo al sistema trabajar en un esquema de detección de borrados en lugar de una detección forzada con mayor probabilidad de error. Por lo tanto, si se requiere un sistema confiable en condiciones bajas de SNR, sería más conveniente usar el sistema con borrado.

## II. PARTE II - ALGORITMOS DE DECISIÓN ITERATIVA

En ésta sección intentaremos abordar mediante la teoría de la información el juego conocido popularmente como wordle. Sea  $X$  el símbolo aleatorio que representa la palabra oculta. Sean  $x_k$  la palabra que ingresamos en el intento  $k$ -ésimo,  $Y_k$  el patrón aleatorio correspondiente y  $X_k$  el conjunto de palabras que concuerdan con los patrones  $y_1, y_2, \dots, y_k$  que hemos observado hasta ahora. El patrón obtenido  $Y_k$  se determina

comparando  $x_k$  con la palabra oculta  $X$  según las reglas del juego: cada letra se clasifica como correcta y en la posición correcta, correcta pero en una posición incorrecta, o no está en la palabra.

Dado que estas reglas son completamente determinísticas, una vez conocidas  $X$  y  $x_k$ , el patrón resultante queda determinado. Es decir, para cada par  $(X, x_k)$  existe un único patrón posible  $Y_k$ . Por lo tanto, aunque  $Y_k$  es una variable aleatoria debido a que la palabra oculta  $X$  es desconocida, puede expresarse como una función de  $X$  y  $x_k$ , es decir

$$H(Y_k|X, x_k) = 0 \quad (27)$$

Pero ¿Qué representa la entropía de la ecuación 27? Intuitivamente es cuanta incertidumbre tenemos sobre la palabra a medida que vamos intentando adivinarla, por cada posición correcta o letra que está pero no se encuentra bien colocada, la misma va decreciendo hasta que al adivinarla se vuelve cero.

Si al momento  $k$ -ésimo de intentar adivinar la palabra, suponemos que la palabra  $X$  se encuentra distribuida uniformemente en  $\mathcal{X}_{k-1}$ , entonces la probabilidad de obtener  $y_k$  dado que la entrada fue  $x_k$  es la misma para toda entrada, la ecuación 28 ilustra lo mencionado:

$$P(X = x_k|Y = y_k) = \frac{1}{|\mathcal{X}|} \forall x \in X_k \quad (28)$$

Lo que nos lleva a que la entropía que tengo sobre la entrada dado que la salida es  $y_k$  es

$$H(X|Y = y_k) = \log(|\mathcal{X}|) \quad (29)$$

Si desarrollamos la definición de entropía condicional, tenemos que

$$H(X|Y_k) = \sum_{y_k \in Y} P(Y = y_k) H(X|Y = y_k) \quad (30)$$

$$H(X|Y_k) = \sum_{y_k \in Y} P(Y = y_k) \log(|\mathcal{X}|) \quad (31)$$

Cómo se vió al comienzo de ésta sección,  $Y_k = f(X, x_k)$  por lo que ésta última ecuación se puede interpretar cómo una esperanza sobre la variable  $X$  de la variable  $Y_k$ , por tanto

$$H(X|Y_k) = \mathbb{E}_{Y_k} [\log(|\mathcal{X}|)] \quad (32)$$

Si buscamos que tan relacionadas se encuentran las variables  $X_k$  e  $Y_k$  podemos representar dicha relación a partir de la información mútua, de ésta manera tenemos que:

$$I(X, Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) \quad (33)$$

$$H(X|Y) = H(X) + H(Y|X) - H(Y) \quad (34)$$

Nuevamente, por lo visto al comienzo de la sección, si se conoce la palabra secreta, y la entrada  $x_k$  se deduce el patrón de colores, y por tanto  $H(Y|X) = 0$ . Entonces para minimizar  $H(X|Y)$  y por tanto que la información mútua

sea máxima, hay que maximizar  $H(Y)$ .

Ahora nos preguntamos ¿Cómo podemos calcular la probabilidad de cada patrón para cierto  $x_k$ ? Para ello basta con estudiar los patrones generados por  $Y_k$ . Por ejemplo, el patrón generado por la palabra que es solución es único y será todo verde, sin embargo, dos palabras distintas pueden generar un mismo patrón de colores, por tanto, podemos describir la probabilidad de obtener un patrón  $y_k$  dado  $x_k$  cómo

$$p(Y_k = y_k|X_k = x_k) = P(y_k = f(X, x_k)) \quad (35)$$

Cómo  $X$  se encuentra uniformemente distribuida en  $X_k$ , llamemos  $M$  a la cantidad de  $x_k$  que generan el patrón  $y_k$ , entonces tenemos que

$$P(y_k = f(X, x_k)) = \frac{M}{|X_k|} \quad (36)$$

Dado esto, podemos calcular la entropía de  $Y_k$  para un intento  $x_k$  cómo

$$H(Y_k|x_k) = - \sum_{y_k} P(Y_k = y_k|x_k) \log(P(Y_k = y_k|x_k)) \quad (37)$$

donde la suma recorre todos los patrones posibles que pueden surgir al comparar  $x_k$  con las palabras en  $X_k$ .

Ahora se nos brinda un script que simula múltiples partidas de Wordle, comenzando siempre con una misma palabra inicial seleccionada previamente. A partir del patrón que se genera tras cada intento, el algoritmo tiene dos opciones para continuar, o bien elige aleatoriamente una palabra del conjunto de candidatas compatibles con el patrón  $Y_k$ , o bien selecciona la que maximice la entropía de  $Y_k$ . En la figura 8 se observa que la estrategia basada en la maximización de la entropía obtiene mejores resultados en promedio en comparación con las palabras al azar, esto se puede observar en la figura 7. Todas las simulaciones comienzan con la misma palabra inicial, ya que algunas palabras contienen letras más frecuentes que otras y de esta manera nos aseguramos que empiezan ambas estrategias en igualdad de condiciones. Luego de varios intentos, notamos que la diferencia de intentos promedios entre maximizar la entropía y elegir al azar una palabra es de aproximadamente 0,7. Se necesitan 0,7 intentos menos si maximizamos la entropía. Al seleccionar palabras al azar entre las disponibles, se necesitan en promedio entre 4,5 y 5 intentos para encontrar la solución, dependiendo de la palabra inicial elegida. Dado que una partida de Wordle permite hasta 6 intentos, este desempeño es comparable al de un jugador humano promedio.

### III. PARTE III - COMPRESIÓN DE LA FUENTE

En esta parte se trabajará con extensiones de la fuente y el algoritmo de Huffman para verificar su capacidad de compresión utilizando los archivos brindados por los docentes del curso.

Comenzamos cuestionándonos el hecho de que la palabra "óptimo" se encuentre entre comillas y nos preguntamos ¿a que

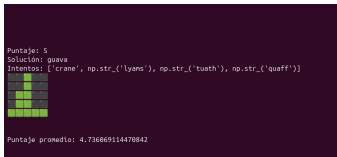


Figura 7. Simulación de wordle sin maximizar la entropía

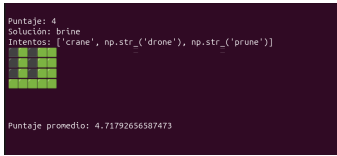


Figura 8. Simulación de wordle maximizando la entropía

se debe esto? Para ello recordemos las hipótesis del código generado mediante el algoritmo de Huffman

“Para cualquier distribución de probabilidad dada, se puede construir un código instantáneo óptimo a través del algoritmo de Huffman.

Dados dos códigos  $C$  y  $C'$ , si  $C$  es un código de Huffman y  $C'$  otro código cualquiera para la misma fuente, entonces  $L(C) \leq L(C')$ ” [1]

Es decir, el código generado a partir del algoritmo de Huffman solamente nos asegura que es óptimo respecto a otros códigos, pero no es el único de ellos, sino que uno de los que minimiza la longitud media.

Luego de esto, utilizando el script de python que se nos brindó, verificamos la compresión de los textos a través de la codificación de Huffman

El código toma un mensaje y lo traduce a binario utilizando la codificación ASCII. Luego en una segunda etapa agrupa los bits según el orden de extensión y calcula la distribución de probabilidad de cada una de esos grupos de bits.

Para entender cómo funciona este algoritmo, veamos el caso del archivo mensaje1.txt, que contiene solamente al carácter A. El código ASCII asociado a éste carácter es 01000001. Al trabajar con una extensión de orden 1 de la fuente, se consideran los bits individuales, por lo que la mayor distribución de ellos será para los 0's, ya que hay 6 ceros y 2 unos, generando

una probabilidad de 0,75 y 0,25 respectivamente.

Al realizar una extensión de orden 2, se generan grupos de a 2 bits, por lo que para la palabra A se realiza la agrupación de todas las posibles cadenas de largo 2, es decir 00;01;10;11. En éste caso, el símbolo 00 aparece dos veces, al igual que el 01, por lo tanto ámbos tendrán probabilidad 0,5.

Si continuamos con la lógica de extensión de fuente, en el orden 4 solamente dos símbolos aparecen en la codificación de A y son el 0100 y 0001 ambos con misma probabilidad. Por último, para el caso de extensión de orden 8, existe un único símbolo que representa a la codificación de A y lo hace con probabilidad 1. Las figuras 9, 10, 11 y 12 ilustran lo mencionado

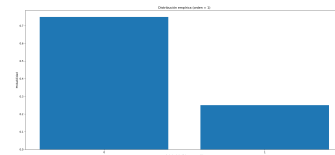


Figura 9. Distribución de probabilidad para extensión de la fuente de orden 1

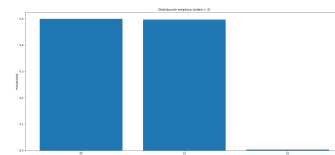


Figura 10. Distribución de probabilidad para extensión de la fuente de orden 2

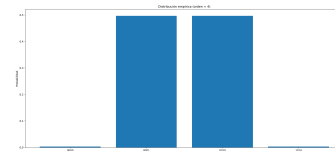


Figura 11. Distribución de probabilidad para extensión de la fuente de orden 4

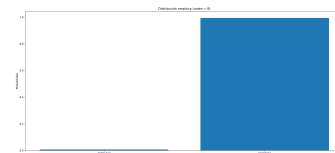


Figura 12. Distribución de probabilidad para extensión de la fuente de orden 8

| Archivo utilizado | Entropía por bit | Entropía Fuente Codificada | Extensiones |
|-------------------|------------------|----------------------------|-------------|
| adams             | 0.9937269        | 0.9937269                  | 1           |
| adams             | 0.9782744        | 0.9791655                  | 2           |
| adams             | 0.8425417        | 0.9952297                  | 4           |
| adams             | 0.5539221        | 0.9952713                  | 8           |
| mensaje1          | 0.8112781        | 0.8112781                  | 1           |
| mensaje1          | 0.5132066        | 0.9196181                  | 2           |
| mensaje1          | 0.2632066        | 0.9196103                  | 4           |
| mensaje1          | 0.0066033        | 0.0525637                  | 8           |
| mensaje2          | 1.0000000        | 1.0000000                  | 1           |
| mensaje2          | 0.999541         | 0.9999662                  | 2           |
| mensaje2          | 0.9994007        | 0.9995912                  | 4           |
| mensaje2          | 0.9897617        | 0.9986161                  | 8           |

Cuadro I

COMPARACIÓN ENTROPÍA POR BIT DE FUENTE VS ENTROPÍA POR CODIFICACIÓN DE HUFFMAN

Al analizar las longitudes de los códigos de Huffman generados para los distintos archivos de texto, notamos que en los textos donde los símbolos se encuentran distribuidos más equilibradamente, las longitudes de los códigos tienden a mantenerse en un rango acotado, con palabras que no reciben codificación extremadamente corta o larga. Por otra parte, en

| Archivo  | Huffman (bits) | Huffman (Bytes) | Zip (Bytes) |
|----------|----------------|-----------------|-------------|
| adams    | 8428           | 1053.5          | 1188        |
| mensaje1 | 168            | 21              | 181         |
| mensaje2 | 16281          | 2035.1          | 2222        |

Cuadro II

COMPARACIÓN DE TAMAÑO DE LOS ARCHIVOS MEDIANTE LOS DISTINTOS MÉTODOS DE COMPRESIÓN

el archivo de mensaje1.txt, al contener sólo al carácter A, se observa un comportamiento diferente. Si realizamos la extensión de orden 8 de la fuente y miramos las primeras 16 salidas, vemos cómo el código correspondiente al símbolo A tiene longitud 1, ya que es el más probable, sin embargo, como los otros 255 símbolos no pertenecen al archivo, el algoritmo le asigna al azar un código correspondiente, que variará su longitud hasta 255, pero que no tendrá peso alguno en el largo medio del código ya que la probabilidad de aparición es 0.

Para el caso del archivo mensaje2.txt, los distintos símbolos aparecen con una tasa similar, por lo que los códigos que asigna Huffman no varía tanto. Si nuevamente hacemos la extensión de orden 8 de la fuente, vemos que la asignación de los códigos y sus largos varían entre longitud 7 y 10 bits. Para el último caso el archivo adams.txt el mismo se encuentra escrito en inglés, por lo que es de esperar que la tasa de aparición de las letras en el texto corresponda con la frecuencia de aparición de las letras en el inglés, por lo que se espera un código con longitudes variables, pero que no está en los casos extremos de los archivos anteriores. En este caso, el símbolo más probable es el espacio (codificación 00100000 en ASCII) seguidos por la n y la r respectivamente.

La codificación de Huffman refleja la frecuencia de aparición de éstas letras, por ejemplo, hay una palabra de longitud 3, nueve palabras de código de longitud 4 y cuatro de longitud 5, así hasta completar la longitud 12. A partir de aquí ya no quedan símbolos con probabilidad positiva, por lo que el algoritmo le asigna codificación al azar y longitudes que llegan hasta 217 a los símbolos restantes con el fin de completar el árbol, sin la necesidad de que los mismos representen información real.

A continuación se presenta una tabla (ver tabla II) que busca comparar la compresión del archivo mediante el algoritmo de Huffman y zip.

El algoritmo de Huffman obtiene una mejor compresión que zip en todos los casos si se considera solo el tamaño codificado. Esta comparación no es justa, ya que zip incorpora metadatos, cabeceras y otras técnicas de compresión, lo que explica su mayor tamaño, pero también su robustez para comprimir.

#### IV. PARTE IV - CODIFICACIÓN PARA CORRECCIÓN DE ERRORES

##### IV-A. Ejercicio 9.4

IV-A1. Parte a: Sea un código  $C$  lineal definido por la siguiente matriz de paridad

$$\mathbb{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \quad (38)$$

Si recibimos la palabra 110110 y se comete sólo un error ¿cuál es la palabra de código enviada?

Para responder éste apartado, recordamos que una palabra de código es válida si y sólo si, el síndrome asociado al recibirla es cero, ésto es

$$r\mathbb{H}^T = (c + s)\mathbb{H}^T = 0 \iff s\mathbb{H}^T = 0 \quad (39)$$

Por lo que basta observar el síndrome asociado a la palabra recibida, si llamamos  $r$  a la palabra recibida y la multiplicamos por la matriz de paridad

$$r\mathbb{H}^T = [1 \ 1 \ 0 \ 1 \ 1 \ 0] \mathbb{H}^T \quad (40)$$

$$= [1 \ 1 \ 0] \quad (41)$$

ésto nos indica que existe un error asociado a la palabra recibida, cómo se trata de un sólo error y además el vector resultante es una columna de  $\mathbb{H}$  entonces podemos concluir que el error es cometido por el bit asociado a la segunda columna, y por tanto la palabra enviada fue

$$c = [1 \ 0 \ 0 \ 1 \ 1 \ 0] \quad (42)$$

IV-A2. Parte b: Considere un código  $(7, 4)$  con matriz generadora

$$\mathbb{G} = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (43)$$

(b.1) Encuentre todas las palabras del código

Como se trata de una matriz generadora de dimensiones  $(7, 4)$  tenemos que el código se construye tomando palabras de 4 bits y generando palabras de código de 7 bits, por tanto habrán  $2^4 = 16$  palabras de código generadas, para hallarlas planteamos todas las combinaciones posibles de 0's y 1's y lo multiplicamos por la matriz generadora, obteniendo así

$$\text{span}(G) = \left\{ \begin{array}{cccc} 0000000 & 1100001 & 0110010 & 1010011 \\ 1010100 & 0110101 & 1100110 & 0000111 \\ 0101100 & 1001101 & 0011110 & 1111111 \\ 1111000 & 0011001 & 1001010 & 0101011 \end{array} \right\}. \quad (44)$$

(b.2) ¿Cuál es la distancia mínima del código?

La distancia mínima del código puede determinarse cómo la mínima cantidad de 1's asociados a una palabra de código,

en nuestro caso, si observamos la ecuación 44 observamos que la distancia es  $D = 3$

(b.3) Encuentre la matriz de paridad  $H$  del código

La matriz de paridad  $H$  se puede construir de forma sistemática a partir de la matriz generadora  $G$ . Si escribimos a  $G$  en su forma sistemática tenemos que

$$G = [I_k | P] \quad (45)$$

entonces podemos obtener la matriz de paridad traspuesta cómo

$$H^T = \begin{bmatrix} P \\ I_{n-k} \end{bmatrix} \quad (46)$$

De ésta manera, y entendiendo que a través de transformaciones lineales podemos escalarizar la matriz generadora para llevarla a su forma sistemática, de manera que si realizamos las operaciones  $\text{fila } 1 = \text{fila } 1 + \text{fila } 2$  y luego intercambiamos las columnas en el siguiente orden  $1 \leftrightarrow 4, 2 \leftrightarrow 5, 3 \leftrightarrow 6, 4 \leftrightarrow 7$  obtenemos la matriz

$$G' = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} \quad (47)$$

la matriz 47 genera un código equivalente al de la matriz 43, ya que el hecho de haber intercambiado columnas, me genera una permutación de los bits de paridad, pero la información es la misma. Una vez obtenida la matriz en su forma sistemática, podemos obtener  $H$  fácilmente

$$H = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix} \quad (48)$$

(b.4) Encuentre el síndrome del vector recibido  $R = [1101011]$  ¿Qué conclusiones puede extraer respecto de la secuencia enviada?

Repetiendo lo hecho en la parte a de éste ejercicio, podemos obtener el síndrome asociado al vector recibido, pero esta vez con la matriz de paridad  $H$

$$RH^T = [1 \ 1 \ 0 \ 1 \ 1 \ 0] H^T \quad (49)$$

$$= [0 \ 0 \ 0] \quad (50)$$

Por lo que en este caso, el síndrome asociado es cero, lo que implica que no hubo error en la recepción de la palabra de código, o hubieron errores pares tales que convirtieron a la palabra enviada en otra palabra de código válida.

(b.5) Bosqueje una implementación como un filtro de éste código

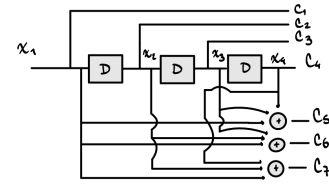


Figura 13. Diagrama de bloques para el código (7,4)

Para poder representar éste código a través de la implementación de un filtro, observemos cual es la salida asociada a una palabra de 4 bits genérica, esto es

$$C = (x_1, x_2, x_3, x_4)G \quad (51)$$

$$= (x_1, x_2, x_3, x_4, p_1, p_2, p_3) \quad (52)$$

donde

$$p_1 = x_1 \oplus x_3 \oplus x_4 \quad (53)$$

$$p_2 = x_1 \oplus x_2 \oplus x_3 \quad (54)$$

$$p_3 = x_1 \oplus x_2 \oplus x_4 \quad (55)$$

por lo que el filtro lo podemos diseñar cómo se ve en el diagrama 13

#### IV-B. Ejercicio 9.6

IV-B1. Parte a: Considere el código convolucional definido por la función en transformada-D:  $[1, 1 + D + D^2, 1 + D^2]$ , el cual es un código de tasa  $\frac{1}{3}$

(a.1) Dibuje un diagrama de tipo shift-register y un diagrama de estados de éste código

Tanto el diagrama de tipo shift register y de estados pueden observarse en la figura 14

(a.2) Dibuje un diagrama de Trellis de 4 transiciones de estados del código anterior

El diagrama de Trellis se ve ilustrado en la figura 15

(a.3)

Para aplicar el algoritmo de Viterbi, debemos analizar la secuencia recibida  $R = 111, 100, 001, 011$  paso a paso sobre el diagrama de Trellis (figura 15).

Para cada etapa calculamos la métrica de rama (distancia de Hamming respecto a la salida esperada) y actualizamos la métrica de camino acumulada (la suma de las distancias).

Recibido:  $R_1 = 111, R_2 = 100, R_3 = 001, R_4 = 011$

Etapas 1 ( $R_1 = 111$ ):

- De  $S_0 \rightarrow S_0$ : Salida 000, dist=3
- De  $S_0 \rightarrow S_1$ : Salida 111, dist=0
- Sobrevivientes:  $S_0$  (dist 3),  $S_1$  (dist 0).

Etapas 2 ( $R_2 = 100$ ):

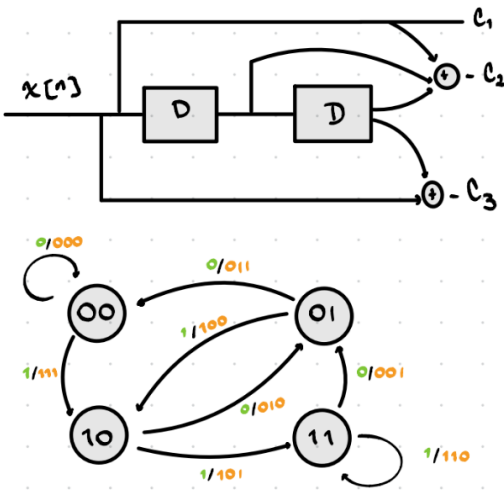


Figura 14. Diagrama shift-register y de estados para el código dado

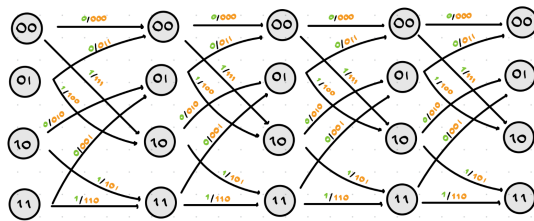


Figura 15. Diagrama de trellis para n = 4

- Desde  $S_1$ :  $S_1 \rightarrow S_2$  (Salida 011, dist Hamming respecto a 100 es 2). Total:  $0 + 2 = 2$ .
- Desde  $S_1$ :  $S_1 \rightarrow S_3$  (Salida 100, dist Hamming respecto a 100 es 0). Total:  $0 + 0 = 0$ .
- Sobrevivientes:  $S_2$  (dist 2),  $S_3$  (dist 0).

Etap 3 ( $R_3 = 001$ ):

- Desde  $S_2$ :  $S_2 \rightarrow S_0$  (Salida 011, dist 2). Total:  $2 + 2 = 4$ .
- Desde  $S_3$ :  $S_3 \rightarrow S_2$  (Salida 000, dist 1). Total:  $0 + 1 = 1$ .
- Sobrevivientes:  $S_0$  (dist 4),  $S_2$  (dist 1).

Etap 4 ( $R_4 = 011$ ):

- Desde  $S_2$ :  $S_2 \rightarrow S_0$  (Salida 011, dist 0). Total:  $1 + 0 = 1$ .
- Desde  $S_2$ :  $S_2 \rightarrow S_1$  (Salida 100, dist 2). Total:  $1 + 2 = 3$ .
- Final: El camino de mínima distancia acumulada termina en  $S_0$  con un valor de 1.

Resultado:

Rastreando hacia atrás desde el estado final ( $S_0$  en la etapa

4), el camino más probable es:  $S_0 \xrightarrow{1} S_1 \xrightarrow{0} S_3 \xrightarrow{0} S_2 \xrightarrow{0} S_0$

Por lo tanto, la secuencia de entrada más probable es 1100.

En la figura 16 se muestra el camino obtenido más probable.

#### IV-C. Ejercicio 9.7

IV-C1. Parte a: Analizar el archivo ejercicio 7 a.grc y calcular el BER para los valores de  $snr = 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5$

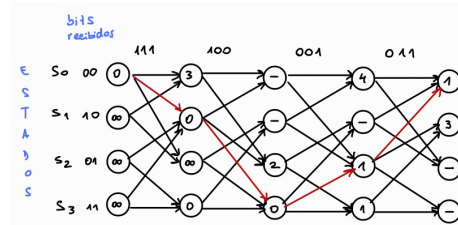


Figura 16. Resultado algoritmo de Viterbi

| SNR | BER    |
|-----|--------|
| -5  | -0.41  |
| -4  | -0.465 |
| -3  | -0.505 |
| -2  | -0.609 |
| -1  | -0.762 |
| 0   | -0.966 |
| 1   | -1.27  |
| 2   | -1.75  |
| 3   | -2.48  |
| 4   | -3.52  |
| 5   | -37.9  |

Cuadro III  
RELACIÓN SNR VS BER PARTE A

Utilizando GNU Radio para el sistema de la parte A se obtuvo la tabla III al ir variando el valor de SNR. Cabe destacar que el delay usado fue 6, que fue el que minimizó el BER para un caso de ruido despreciable.

IV-C2. Parte b: Analizar el archivo ejercicio 7 b.grc y calcular el BER para los valores de  $snr = 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5$ . Graficar el BER de la parte a) y b) y compararlos.

Para tomar las medidas de la tabla IV se procedió de manera similar a la parte A pero se utilizó delay de 3 que fue el que minimizaba el BER.

Luego de obtener esta serie de medidas podemos obtener la figura 17 que muestra las curvas de BER que surgen de las partes A y B a partir de los valores de SNR:  $[5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5]$ . Cabe destacar que el BER de la parte B se mantiene igual o por debajo del BER de la parte A. Esto tiene sentido teórico, ya que el sistema de la parte B utiliza codificación convolucional, combinada con la decodificación mediante el algoritmo de Viterbi, esto maximiza la probabilidad de decidir la secuencia correcta

| SNR | BER   |
|-----|-------|
| -5  | -0.50 |
| -4  | -0.59 |
| -3  | -0.73 |
| -2  | -0.96 |
| -1  | -1.34 |
| 0   | -1.96 |
| 1   | -2.83 |
| 2   | -4.20 |
| 3   | -6.50 |
| 4   | -24.1 |
| 5   | -37.9 |

Cuadro IV  
RELACIÓN SNR VS BER PARTE B



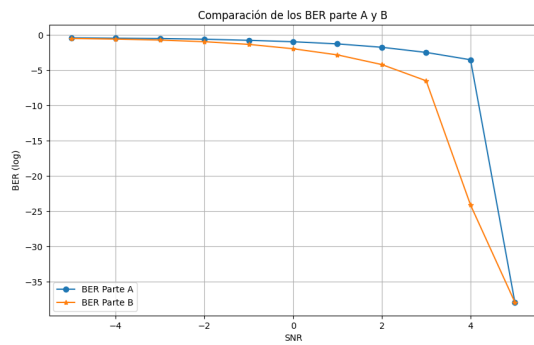


Figura 17. BER de la parte A y BER de la parte B

mediante la búsqueda del camino de mínima distancia. La redundancia introducida por el codificador permite que el decodificador recupere la información original incluso ante la ocurrencia de errores en el canal, aprovechando la distancia libre del código para alcanzar una ganancia de codificación que desplaza la curva de BER hacia abajo.

*IV-C3. Parte c: Analizar el archivo ejercicio 7 c.grc, y utilizar un archivo WAV que tenga en su equipo. Aumentar la potencia de ruido hasta el punto en que se comienza a sentir ruido y registrar estos dBs. Modificar el archivo anterior para utilizar un codificador de Trellis y decodificador de Viterbi Combo. Volver a calcular ahora el punto en el que comienza a sentirse ruido y registrar los db. ¿Cuál es la ganancia en db en la calidad percibida del audio por utilizar un código de corrección de errores?*

Para ésta parte se decidió utilizar el archivo RATM.wav muestreado a una tasa de 48kHz y se verificó que para un volumen de 0dB a partir de  $-6,7dB$  de potencia de ruido, es decir una  $SNR = 6,7dB$ , se comienza a distorsionar el audio de la señal.

Luego, se utilizó el codificador de Trellis y decodificador de Viterbi Combo y para el mismo nivel de volumen, se registró que a partir de una potencia de ruido de  $-3,0dB$  la señal comenzó a distorsionarse, por lo que observamos una mejora de 3dB aproximadamente respecto al decodificador anterior.

#### REFERENCIAS

- [1] Facultad de Ingeniería, Universidad de la República (UdelaR), "Codificación de fuentes," curso Introducción a la Teoría de la Información, Montevideo, Uruguay. Disponible: [https://eva.fing.edu.uy/pluginfile.php/74893/mod\\_resource/content/0/iti-codificacion\\_de\\_fuentes-handout.pdf](https://eva.fing.edu.uy/pluginfile.php/74893/mod_resource/content/0/iti-codificacion_de_fuentes-handout.pdf)